ASRP: Adaptive Support-Based Response Prediction for Test-Time AI Capability Evaluation

Yuhan Chiu
National Taiwan University
Taipei, Taiwan
yhn.chiu@gmail.com

Coauthor name
Affiliation placeholder
Address placeholder
coauthor@example.com

Abstract

A single ability estimate can miss a subject's relative strengths across item types, while local adaptation from very few labels can overreact to individual outcomes. We present **ASRP (Adaptive Support-Based Response Prediction)**, which addresses this tension by shifting from public-data priors and content-based Rasch estimation at low label budgets to support-conditioned kernel residuals as evidence accumulates. A fixed budget-dependent ensemble controls this transition, shrunk topic corrections adjust systematic errors, and a uniform random streaming rule supplies the revealed labels. Across 30 public-data draws following the documented formative protocol, ASRP achieves mean Brier ALC 0.1895 versus 0.2285 for Empirical Mean with identical randomly acquired labels. This matched-evidence comparison evaluates the complete predictor.

1 Introduction

The Predictive AI Evaluation Competition (PAIEC) asks for an AI system's success probability on an individual item with at most 31 revealed labels per subject–benchmark pair. It requires both zero-label prediction and adaptation from sparse responses. Our target is query-level success, following the distinction from confidence in a particular generated answer made by Yang et al. [2026].

Shared structure and local evidence address different limitations. Item response theory models subject ability and item difficulty [Lord, 1980], while amortized evaluation predicts difficulty from content [Truong et al., 2025]. This shared structure supports cold-start prediction, but a scalar ability cannot express every relative strength across item types, and frozen estimates may transfer imperfectly to new benchmarks. Conversely, local corrections learned from one or two labels can overreact to individual outcomes. ASRP therefore combines prior-based estimation with regularized support-driven adaptation, increasing the latter's weight as evidence grows.

ASRP couples two prediction branches. A Rasch hybrid combines difficulty from frozen MiniLM representations [Wang et al., 2020], updated subject ability, and metadata-informed smoothing. A kernel probe learns item-dependent residuals around a smoothed subject baseline from revealed responses. **Budget-dependent mixing** controls their relative influence. Shrunk topic calibration corrects systematic public-data residuals, while uniform random acquisition selects labels from the candidate stream. This integration is accompanied by released source selections, matched splits, predictions, and acquisition traces for reproducible evaluation.

Matched randomly acquired labels isolate the predictor comparison. Across 30 complete-group public draws, ASRP achieves mean Brier ALC 0.1895 versus 0.2285 for Empirical Mean. All six mean budget scores favor ASRP in this public simulation. Public fitting and evaluation overlap, and transfer to the private competition pool remains unverified for the current submission.

Preprint.

2 Competition Setting and Data

2.1 Prediction interface and metrics

At runtime the platform supplies a pair of dictionaries $x = (s, i)$ describing a subject and an item, together with a list of previously acquired binary observations. The predictor returns a finite probability in $[0, 1]$. Eligible subject–benchmark groups have at least 80 distinct items. The platform creates a fixed acquisition/evaluation split and evaluates the same evaluation items at the **six label budgets**

$$n \in \{0, 1, 3, 7, 15, 31\}.$$

Acquisition items are never scored and evaluation outcomes are never revealed. The optional acquisition hook receives the current candidate, acquired labels, and remaining-label context; the interface can also supply a current prediction when requested. ASRP uses the current candidate and remaining-budget context for its reproducible random draw.

For a budget n and a subject–benchmark group with N_{eval} evaluated responses, the Brier score is the mean squared error between the predicted probability p and the binary outcome y:

$$B_n = \frac{1}{N_{\text{eval}}} \sum_{j=1}^{N_{\text{eval}}} (p_j - y_j)^2.$$

The group-level Brier ALC is the normalized area on the logarithmic budget axis and is equivalent to

$$\text{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}.$$

The official score averages these group-level ALC values equally across subject–benchmark groups. **Lower is better.** All reported local tables use this same weighting.

2.2 Public data and frozen artifacts

The predictor uses frozen artifacts trained and calibrated from the public `aims-foundations/measurement-db` release [AIMS Foundations, 2026]. Public responses provide subject priors and metadata statistics, while item content supplies semantic features. The local evaluation samples from 10,208 eligible subject–benchmark groups in 92 sources within a fixed inventory of 142 binary sources; Section 4 describes selection and splitting. Appendix B lists the evaluated sources. Upstream metadata and licenses remain authoritative.

The subject prior and metadata statistics are frozen public-response aggregates. Their runtime use and fallback rules are specified in Appendix A; reproducing the reported scores uses the released weights without refitting them. MiniLM is used as a pretrained encoder; the packaged encoder is `sentence-transformers/all-MiniLM-L6-v2`, based on MiniLM [Wang et al., 2020]. Its weights are **frozen at runtime**. The difficulty model, priors, kernel configuration, calibration statistics, and cluster centroids are prepared offline and bundled with the submission; kernel residual coefficients are fitted online from revealed labels. The current submission uses **no external LLM or web search API during scoring**. At runtime, the program uses only the supplied subject and item attributes, anonymous benchmark identifiers, and labels revealed within the allowed budget; it does not access hidden evaluation outcomes.

3 Proposed Method: ASRP

For a query subject s and item i in benchmark b, let $\mathcal{D}_b$ be the revealed support and $\mathcal{D}_s$ its rows for the current subject. Write $n = |\mathcal{D}_s|$ and $S_s = \sum_{r \in \mathcal{D}_s} y_r$. All predictions below concern this query; their arguments are suppressed when unambiguous. Figure 1 shows the computation. Appendix A gives the shared notation, numerical details, and corresponding code functions.

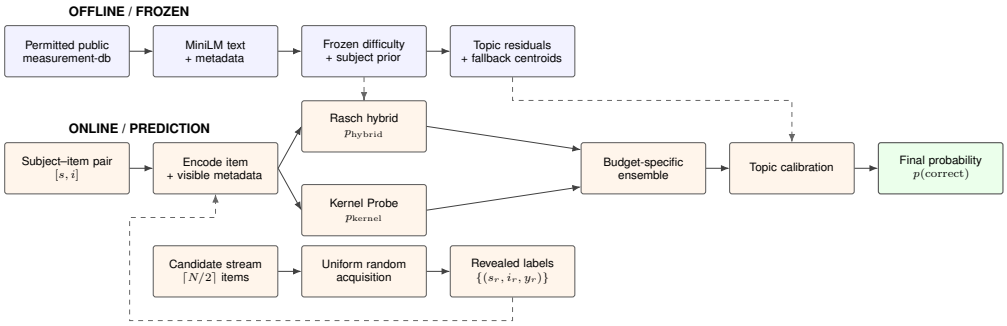


Figure 1: ASRP overview. Frozen public-data artifacts supply difficulty, prior, and calibration estimates. Revealed labels adapt the Rasch hybrid and kernel probe before budget-specific ensembling and topic correction. For a complete group of N items, the candidate pool contains $\lceil N/2 \rceil$ items. Uniform random acquisition supplies nested label prefixes.

3.1 Item representation and amortized difficulty

MiniLM encodes the normalized item text as $e_i \in \mathbb{R}^{384}$. Six deterministic numeric text features form $z_i \in \mathbb{R}^6$. A frozen feed-forward network standardizes their concatenation and predicts a clipped difficulty:

$$d_i = \text{clip}(f_\phi([e_i; z_i]), -2, 2)\,.$$

The network has dimensions 390–128–32–1 and ReLU hidden layers. This amortized estimate supplies a difficulty for an unseen item without fitting an item-specific parameter online. Appendix A.1 specifies text limits, pooling, and standardization.

3.2 Public prior and Rasch update

A frozen subject offset o_s, looked up by normalized model name, defines a prior mean and initial ability:

$$\mu_s = \text{clip}(0.5 + 0.5o_s, 0.05, 0.95), \qquad \theta_{0,s} = \text{logit}(\mu_s).$$

An unknown name has $o_s = 0$. With revealed labels, clipped Newton–Raphson iterations approximate the **maximum a posteriori (MAP) estimate** under a Bernoulli Rasch likelihood and Gaussian ability prior $\mathcal{N}(\theta_{0,s}, v_\theta)$, with $v_\theta = 2$. The resulting ability $\hat{\theta}_s$ gives

$$p_{\text{Rasch}} = \sigma(\hat{\theta}_s - d_i), \qquad \sigma(t) = (1 + e^{-t})^{-1}.$$

At $n = 0$, $\hat{\theta}_s = \theta_{0,s}$. Separately, a query-specific metadata prior $p_{\text{meta}}(s, i)$ combines frozen subject and visible-feature counts. The hybrid smooths observed successes toward this prior and blends the result with the Rasch probability:

$$p_{\text{smooth}} = \frac{S_s + \kappa p_{\text{meta}}(s,i)}{n + \kappa}, \qquad p_{\text{hybrid}} = (1 - \eta_n)p_{\text{smooth}} + \eta_n p_{\text{Rasch}}, \qquad \kappa = 4.$$

Here η_n is the frozen Rasch mixing weight. Appendix A.2 gives the prior lookup and Newton updates; Appendix A.4 lists the six mixing weights.

3.3 Support-conditioned kernel probe

The kernel branch adds an item-dependent correction to a conservative subject baseline. It encodes visible text, metadata, and interactor descriptions into semantic vectors $\mathbf{h}_i$ and lexical vectors $\boldsymbol{\ell}_i$. A locally normalized similarity kernel K is fitted using only the support features in $\mathcal{D}_b$; the query does not change this geometry.

For each supported subject s, let n_s and S_s be its revealed-label and success counts. A two-component Beta mixture has concentrations $(c_1, c_2) = (1, 8)$, prior masses $(\pi_1, \pi_2) = (0.3, 0.7)$, and posterior weights ω_{sk}. Its mean is

$$b_s = \sum_{k=1}^{2} \omega_{sk} \frac{c_k \mu_s + S_s}{c_k + n_s}.$$

Subject-centered kernel ridge regression, with penalty $\lambda_K = 0.3$, fits residual coefficients γ from the revealed labels. With query-to-support similarities $\mathbf{k}_i$ and subject mean fitted contribution $\bar{f}_s$, the prediction is

$$p_{\text{kernel}} = \text{clip}\big(b_s + \mathbf{k}_i^\top \gamma - \bar{f}_s, 0, 1\big)\,.$$

Appendix A.3 defines K, the Beta weight update, and the centered linear solve. If the current subject has no revealed labels, this branch returns μ_s. The implementation supports multiple subjects within a benchmark; the evaluator supplies evidence across all sampled groups, and this branch filters it to the current benchmark.

The ensemble gives the kernel weights $w_n = (0, 0, 0.2, 0.4, 1, 1)$ at budgets $(0, 1, 3, 7, 15, 31)$. Thus the Rasch hybrid supplies the cold-start prediction, while a larger support set allows the local residual to receive more weight. These are design choices encoded in the released configuration.

3.4 Topic calibration

A frozen topic correction $\Delta_n(i)$ adjusts the ensemble using the first matching topic entry in visible metadata. Without a match, the model uses the closest of eight frozen embedding centroids. Count-based shrinkage and caps limit the adjustment to 0.06 for explicit topics or 0.03 for clusters. Missing statistics give zero correction.

The budget-specific ensemble is followed by topic correction:

$$p_{\text{final}} = \text{clip}((1 - w_n)p_{\text{hybrid}} + w_n p_{\text{kernel}} + \Delta_n(i), 0, 1)\,.$$

Appendix A.4 gives the exact correction and lookup rules.

3.5 Random acquisition

The submission uses uniform random streaming acquisition. With L labels and R candidates remaining, including the current candidate, the acquisition probability is

$$\pi^{\text{acq}} = \min(1, L/R).$$

A deterministic pseudorandom draw makes decisions reproducible. The rule stops acquiring when $L = 0$ and acquires every remaining candidate when $R \leq L$ and $L > 0$. It uses no warm-up, feature-based priority, or outcome-dependent selection. Appendix A.5 specifies the implementation. The term "Adaptive" in ASRP refers to prediction updates from revealed support, not to label selection.

4 Experimental Setup

We compare ASRP and Empirical Mean using identical randomly acquired labels on public-data draws. Code, frozen artifacts, and exact reproduction commands are provided in Appendix C.

4.1 Public formative simulation

Complete groups and repeated draws. The pinned public snapshot contains 10,208 eligible subject–benchmark groups in 92 sources, each with 80–1,000 distinct items. We predeclare **30 draw seeds**. Each draw selects complete groups under a **total 1,000-pair cap**, preferring less-represented benchmarks and subjects with random tie-breaking; groups that do not fit are skipped. The draws cover 80 distinct groups from 55 sources, with 1–5 groups and 922–1000 pairs per draw.

Fixed pools and all responses. One snapshot seed fixes each group's 50/50 item split and candidate order across draws. All runs of an item stay together. Acquisition reveals one fixed recorded response per selected item; **all recorded evaluation responses are scored**, reusing a prediction for identical visible inputs. Across draws, this yields 14,629 evaluation-item occurrences and 665,052 response occurrences per policy, including repeated appearances of groups.

Matched evidence and checkpoint barriers. ASRP and Empirical Mean use the same draws, pools, and uniformly sampled labels. One trajectory supplies nested prefixes. **Every group completes a checkpoint before evaluation**; each call receives all evidence revealed across groups, excluding future labels and evaluation outcomes. Released diagnostic variants use these same labels and frozen artifacts.

Alignment. These constraints follow the public rules and starting kit. Private coordinator details are unavailable, so benchmark-first tie-breaking, group scheduling, and acquisition-response selection remain explicit local choices (see the released protocol in Appendix C). This is a **public simulation of the documented design**, not an exact private-sampler replay or a source-held-out test; public fitting and calibration overlap the evaluation release.

4.2 Baseline and score aggregation

We use the **Empirical Mean predictor** from the official PAIEC repository [Truong et al., 2026]:

$$p_{\mathrm{EM}} = \begin{cases} 0.5, & n = 0, \\ \frac{1}{n}\sum_{k=1}^{n} y_k, & n > 0. \end{cases}$$

It uses only the current subject–benchmark group's acquired labels, without item features or a learned prior. Both predictors receive the same labels selected by the random streaming rule.

We average response losses within each group, then weight groups equally within each draw and average over 30 draws. We report the mean, sample standard deviation, and range of draw scores, comparing methods on identical draws.

5 Results and Analysis

5.1 Public draw results

Across 30 public-data draws, **ASRP reaches mean Brier ALC** 0.1895, compared with 0.2285 for Empirical Mean using identical randomly acquired labels (Table 1). Each draw contains complete groups under the 1,000-pair cap; Table 2 reports score variation across sampled group compositions.

Predictor (Random acquisition)	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
Empirical Mean	0.2500	0.3439	0.2161	0.1909	0.1794	0.1745	0.2285
ASRP	0.2111	0.2032	0.1924	0.1830	0.1778	0.1707	0.1895

Table 1: Public formative-simulation Brier scores: equal groups within each draw, then equal weight across 30 draws. Both predictors share evaluation pools and randomly acquired labels. Lower is better.

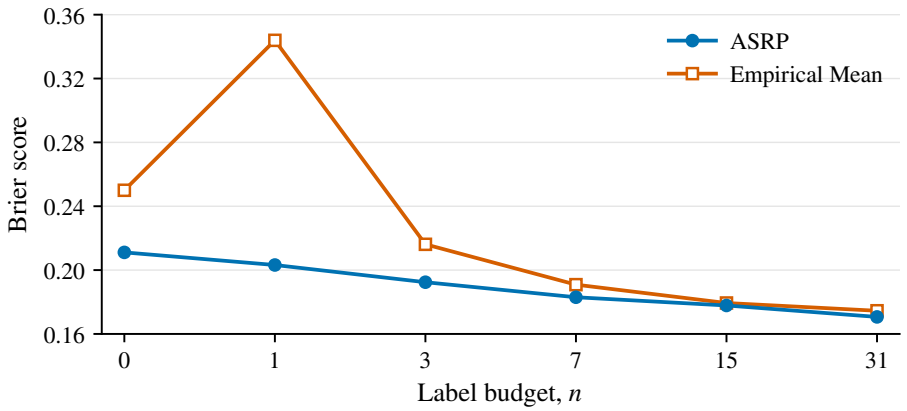


Figure 2: Mean Brier scores across 30 public formative-simulation draws with matched Random acquisition, plotted on the $\log_2(1+n)$ budget axis. Lower is better. Values match Table 1.

Predictor (Random acquisition)	Mean ALC	SD	Minimum	Maximum
Empirical Mean	0.2285	0.0701	0.0286	0.3613
ASRP	0.1895	0.0533	0.0260	0.2549

Table 2: Distribution of group-averaged Brier ALC across 30 public draws. SD and range describe changing sampled group compositions under fixed within-group pools.

5.2 Prediction with matched evidence

Matched-evidence comparison. With randomly acquired evidence held fixed, ASRP changes ALC by -0.0391 relative to Empirical Mean and scores lower in 28 of 30 draws. The comparison measures the complete predictor under the same label-selection rule. Draw-level SD describes variability in the sampled public group compositions.

5.3 Component ablations

We disable Rasch ($\eta_n = 0$), the kernel ($w_n = 0$), or topic calibration individually, keeping the same draws and randomly acquired labels without retraining or retuning.

Variant	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC	ΔALC
ASRP	0.2111	0.2032	0.1924	0.1830	0.1778	0.1707	0.189458	—
w/o Rasch	0.2112	0.2027	0.1926	0.1837	0.1778	0.1707	0.189548	+0.000091
w/o kernel	0.2111	0.2032	0.1925	0.1831	0.1781	0.1747	0.189949	+0.000491
w/o topic	0.2172	0.2065	0.1924	0.1824	0.1771	0.1703	0.190465	+0.001007

Table 3: Component ablations over 30 matched Random draws. Positive ΔALC (variant minus ASRP) indicates worse performance.

All removals increase mean ALC, with different budget-specific effects. Topic calibration contributes most at budget 0 (Brier difference on removal: $+0.0062$), and the kernel at budget 31 ($+0.0040$). Rasch has a small mean ALC benefit (0.000091) and no effect at budgets 15 and 31, where its branch has zero weight. These benefits are not uniform across budgets or draws.

6 Limitations and Discussion

Evaluation scope and generalization. Public fitting and calibration overlap the evaluation release, so local gains may be optimistic. The 30 draws can reuse groups and measure variation within this snapshot rather than transfer to unseen subjects or benchmarks, especially at cold start. The simulation also omits multiple-subject draws from the same benchmark and makes assumptions about unavailable coordinator details. Component ablations show small, draw-dependent ALC differences (Section 5.3); consistent benefits across new groups remain unverified.

7 Conclusion

ASRP combines structured initialization with support-conditioned residual adaptation as randomly acquired labels accumulate. ASRP outperforms Empirical Mean on average with matched acquired evidence in the public simulation. The released protocol, frozen artifacts, predictions, and acquisition traces make the predictor comparison reproducible.

A Mathematical and Implementation Details

These equations specify inference with the released frozen artifacts at the six competition budgets. They use the same symbols as Section 3. The reproduction command evaluates these artifacts without repeating their original offline fitting. Throughout, $\mathrm{clip}(x, l, u) = \min(u, \max(l, x))$, applied coordinatewise to vectors, and $\mathrm{norm}(x) = x/\max(\|x\|_2, 10^{-12})$.

Shared notation. A support row r contains a subject s_r, item i_r, and binary outcome y_r. The row collection $\mathcal{D}_b$ contains all revealed responses for benchmark b and has size M. Its subject-specific subset $\mathcal{D}_s$ has n_s rows and S_s successes. For the current query subject, the budget is $n = n_s \le 31$. The kernel can share residual evidence across subjects in the same benchmark; M need not equal n. Runtime subject identity uses the complete subject dictionary, whereas frozen subject-prior lookup uses `normalized_name`.

We distinguish η_n (Rasch mixing weight), w_n (kernel mixing weight), and λ_K (ridge penalty). Beta component index $k \in \{1, 2\}$ has concentration c_k, prior mass π_k, and posterior mass ω_{sk}. Random acquisition uses probability π^{acq} and pseudorandom variate ξ.

A.1 Feature construction for Section 3.1

The Rasch encoder normalizes whitespace, takes the first 6,000 characters of item content, and tokenizes to at most 256 tokens. Masked mean pooling followed by L_2 normalization gives $e_i \in \mathbb{R}^{384}$. The six coordinates of z_i are $\log(1+\text{text length})/10$, digit fraction, non-ASCII fraction, and indicators for code, mathematical notation, and a question mark. Fractions and the question-mark indicator use the first 2,000 characters; code/math markers use the first 6,000. Text length is measured before these truncations.

For stored feature mean $\bar{\mathbf{x}}$ and scale $\boldsymbol{\sigma}_x$, the difficulty network receives $\mathrm{clip}(([e_i; z_i] - \bar{\mathbf{x}}) \oslash \boldsymbol{\sigma}_x, -10, 10)$, where $\oslash$ denotes coordinatewise division. Frozen weights in `difficulty_model.npz` implement the 390–128–32–1 map; the final scalar is clipped to $[-2, 2]$ by `_predict_hybrid`, after `_difficulty` evaluates the network.

The kernel encoder uses a separate text construction containing item content, metadata, and interactor descriptions. It pools up to four evenly spaced windows of 254 content tokens, each with special tokens, normalizes each pooled vector, and normalizes their sum to obtain $\mathbf{h}_i \in \mathbb{R}^{384}$. The lexical vector $\boldsymbol{\ell}_i \in \mathbb{R}^{8192}$ uses signed CRC32 hashing of lowercased tokens and adjacent token bigrams, weights $1 + \log(\text{count})$, and L_2 normalization. Only these semantic and lexical coordinates enter the kernel. See `ml_features.py`, `window_features.py`, and `evidence_features.py` under `src/asrp/`.

A.2 Priors and ability update for Section 3.2

Two distinct frozen priors. The scalar o_s comes from `binary_subject_prior.json`. It defines μ_s and $\theta_{0,s}$ exactly as in Section 3.2. The metadata prior $p_{\mathrm{meta}}(s, i)$ instead uses the success/count tables in `feature_prior.json`; it corresponds to `_feature_prior` and the runtime variable `p_source` in `src/asrp/model.py`. The released source-prior weight is one, prior shift is zero, and metadata-match adaptation weight is zero.

Let (A_0, N_0) be global public successes and counts, and (A_s, N_s) the subject totals. The frozen rates are

$$q_0 = \frac{A_0}{\max(N_0, 1)}, \qquad q_s = \frac{A_s + 8q_0}{N_s + 8}.$$

Unknown subjects use global totals. Their offset o_s is separately zero, so $\theta_{0,s} = 0$, but $p_{\mathrm{meta}}(s, i)$ need not equal 0.5. All released global totals have positive counts.

For each token t in the query metadata with stored global counts, let $q_t = A_t/\max(N_t, 1)$. When subject–token counts exist,

$$q_{st} = \frac{A_{st} + 8q_t}{N_{st} + 8}, \qquad \rho_{st} = \frac{\min(N_{st}, 80)}{\min(N_{st}, 80) + 12}.$$

Otherwise $q_{st} = q_t$ and $\rho_{st} = \min(N_t, 300)/[\min(N_t, 300) + 100]$. For the set $\mathcal{T}(i)$ of matching tokens,

$$p_{\text{meta}}(s, i) = \text{clip}\left(\frac{2q_s + \sum_{t\in\mathcal{T}(i)} \rho_{st} q_{st}}{2 + \sum_{t\in\mathcal{T}(i)} \rho_{st}}, 0.03, 0.97\right).$$

An empty token set gives clipped q_s. Token parsing, deduplication, and stop words follow `_feature_tokens`. The packaged artifact is present; if it were absent, the runtime fallback would be μ_s.

MAP objective and numerical update. Using only $\mathcal{D}_s$, the penalized Bernoulli log-likelihood is

$$\mathcal{L}_s(\theta) = \sum_{r\in\mathcal{D}_s} [y_r \log \sigma(\theta - d_{i_r}) + (1 - y_r) \log(1 - \sigma(\theta - d_{i_r}))] - \frac{(\theta - \theta_{0,s})^2}{2v_\theta}, \qquad v_\theta = 2.$$

Starting at $\theta_{0,s}$, `update_ability` computes

$$\begin{aligned} p_r(\theta) &= \sigma(\text{clip}(\theta - d_{i_r}, -40, 40)), \\ J_s(\theta) &= \sum_{r\in\mathcal{D}_s} (y_r - p_r(\theta)) - (\theta - \theta_{0,s})/v_\theta, \\ \mathcal{I}_s(\theta) &= \sum_{r\in\mathcal{D}_s} p_r(\theta)(1 - p_r(\theta)) + 1/v_\theta, \\ \delta_\theta &= \text{clip}\big(J_s(\theta)/\max(\mathcal{I}_s(\theta), 10^{-6}), -2, 2\big), \qquad \theta \leftarrow \theta + \delta_\theta. \end{aligned}$$

Here J_s is the gradient and $\mathcal{I}_s$ the positive negative-Hessian magnitude, not the Hessian itself. The routine stops after 30 iterations or when $|\delta_\theta| < 10^{-7}$, then clips $\hat{\theta}_s$ to $[-12, 12]$. With no labels it returns $\theta_{0,s}$ directly. The runtime sigmoid also clips its logit to $[-40, 40]$ when computing p_{Rasch}. The hybrid uses at most the first 31 matching labels in acquisition order.

A.3 Kernel calculations for Section 3.3

Support-only geometry and bandwidth. For $M > 1$ rows in $\mathcal{D}_b$, let $\bar{\mathbf{h}}$ and v_j^{feat} be the coordinatewise semantic mean and population variance (divisor M). Define semantic scales t_j and lexical weights ν_ℓ by

$$t_j = \left[v_j^{\text{feat}} + \max(\text{median}_j\, v_j^{\text{feat}}, 10^{-7})\right]^{-1/2}, \qquad \nu_\ell = \log\frac{1 + M}{1 + f_\ell} + 0.1,$$

where $f_\ell = \sum_{r\in\mathcal{D}_b} \mathbf{1}[|\ell_{i_r,\ell}| > 10^{-10}]$. Both support and query use

$$\widetilde{\mathbf{h}}_i = \text{norm}((\mathbf{h}_i - \bar{\mathbf{h}}) \odot \mathbf{t}), \qquad \widetilde{\boldsymbol{\ell}}_i = \text{norm}(\boldsymbol{\ell}_i \odot \boldsymbol{\nu}).$$

For $M = 1$, the mean is zero and both weight vectors are all ones. No query feature enters these fitted statistics.

For support rows r, t, define

$$Q_{rt} = \tfrac{1}{2}\left(\widetilde{\mathbf{h}}_{i_r}^\top \widetilde{\mathbf{h}}_{i_t} + \widetilde{\boldsymbol{\ell}}_{i_r}^\top \widetilde{\boldsymbol{\ell}}_{i_t}\right), \qquad K_{rt} = \exp\left(\frac{\min(Q_{rt} - 1, 0)}{h_K}\right).$$

Take distances $\max(1 - Q_{rt}, 0)$ over unordered off-diagonal pairs, retain those above 10^{-8}, and let d_{med} be their median (or 0.1 if none remain). The bandwidth is $h_K = 4\max(d_{\text{med}}, 10^{-5})$. The vector $\mathbf{k}_i \in \mathbb{R}^M$ uses this same transformation and bandwidth between query i and each support row. These operations correspond to `fit_geometry`, `fit_evidence`, and `predict_fitted` in `src/asrp/project_core.py`.

Beta mixture. For component $k \in \{1, 2\}$, let $\alpha_{sk} = \mu_s c_k$ and $\beta_{sk} = (1 - \mu_s)c_k$, with $(c_1, c_2) = (1, 8)$. The normalized posterior masses are

$$\widetilde{\omega}_{sk} = \pi_k \frac{\text{B}(\alpha_{sk} + S_s, \beta_{sk} + n_s - S_s)}{\text{B}(\alpha_{sk}, \beta_{sk})}, \qquad \omega_{sk} = \frac{\widetilde{\omega}_{sk}}{\sum_{k'=1}^{2} \widetilde{\omega}_{sk'}}, \qquad (\pi_1, \pi_2) = (0.3, 0.7).$$

Here B is the beta function. The implementation evaluates log-beta functions and normalizes by log-sum-exp. These weights give b_s in Section 3.3. If the current subject is absent from the support, `kernel_probe.py` returns μ_s without fitting a residual for that subject.

Centered ridge fit. Let $Z \in \{0,1\}^{M \times J}$ be the membership matrix for the J subjects present in the support, and $D = \mathrm{diag}(n_1, \ldots, n_J)$. Every diagonal count is positive. Let I_M be the $M \times M$ identity and $\mathbf{y} = (y_r)_{r \in \mathcal{D}_b} \in \mathbb{R}^M$. Define

$$H = I_M - ZD^{-1}Z^\top, \qquad K_c = HKH.$$

$$\gamma = H\left[\tfrac{1}{2}(K_c + K_c^\top) + \lambda_K I_M\right]^{-1} H\mathbf{y}, \qquad \lambda_K = 0.3.$$

The target $H\mathbf{y}$ subtracts each subject's empirical mean; the Beta mean b_s enters the final prediction separately. The code evaluates the inverse expression with a linear solve. Symmetrization matches the floating-point implementation; in exact arithmetic K_c is symmetric. The fitted subject contribution is

$$\bar{f}_s = \frac{1}{n_s} \sum_{r \in \mathcal{D}_s} (K\gamma)_r.$$

Substituting b_s, γ, $\mathbf{k}_i$, and $\bar{f}_s$ into Section 3.3 gives the implemented kernel prediction. At $n_s = 0$ this residual expression is bypassed; at $n = 0, 1$ the final ensemble weight w_n is zero.

A.4 Mixing and calibration for Section 3.4

The six released mixing weights are

n	0	1	3	7	15	31
η_n	0.2875	0.3000	0.2875	0.2750	0.2250	0.2375
w_n	0	0	0.2	0.4	1	1

Here η_n maps to `runtime_config.json:lambdas`; w_n is the active `PROBE_WEIGHTS` in `src/asrp/model.py`; and λ_K maps to `probe_config.json:ridge`. They are distinct parameters. The formulas in this report concern the official checkpoint counts; defensive interpolation/fallback behavior at other counts is not used in the experiment.

Let $t = t(i)$ be the first explicit topic key–value entry, in visible metadata order, present in the frozen topic table. If none matches, assign the Rasch embedding e_i to the closest of eight frozen centroids by cosine similarity and use its cluster entry. Write C_t^{cal} for that entry's frozen count and $r_{t,n}$ for its signed correction at budget n. Then

$$\Delta_n(i) = \mathrm{clip}\left(\frac{C_t^{\mathrm{cal}}}{C_t^{\mathrm{cal}} + a_t^{\mathrm{cal}}}\, r_{t,n}, -d_t^{\mathrm{cal}}, d_t^{\mathrm{cal}}\right), \qquad (a_t^{\mathrm{cal}}, d_t^{\mathrm{cal}}) = \begin{cases} (40, 0.06), & \text{explicit topic,} \\ (200, 0.03), & \text{cluster.} \end{cases}$$

If no entry exists, $\Delta_n(i) = 0$. Cluster cosine normalization floors norms at 10^{-8} and resolves ties by the first maximum. At the six checkpoints, the residual array index is the corresponding budget index.

The correction $\Delta_n(i)$ is added after mixing the hybrid and kernel predictions with weight w_n. A final clip to $[0, 1]$ completes `predict`.

A.5 Random acquisition implementation

The hook in `src/asrp/labeling.py` returns false when no labels or candidates remain, and true when all remaining candidates are needed to fill the label allowance. Otherwise it accepts with probability L/R. The pseudorandom value $\xi \in [0, 1)$ is derived deterministically from the current context and input using SHA256; the label is acquired when $\xi < L/R$. This matches the uniform streaming rule used in the public starting kit. The hook requires no model inference and does not inspect outcome values. The maximum allowance is 31 labels; the evaluation checkpoints use nested prefixes of this trajectory.

B Public source inventory

The following 55 public binary source identifiers occur across the 30 public formative draws. They identify the evaluation sources; the benchmark inventory and upstream licenses remain authoritative.

`afrieval`, `algotune`, `alignment_faking`, `babilong`, `benger`, `critic_discernment_game`, `crow`, `devbench`, `editbench`, `egoschema`, `engdesign`, `hallusionbench`, `harmbench`, `healthadminbench`, `helm_bbq`, `helm_thaiexam`, `hle`, `igakuqa119`,

ikp, jailbreakbench, legal_rag_bench, mathvista_mini, medhelm_med_qa, medhelm_medhallu, medhelm_pubmed_qa, mj_bench, mmbench_v11, monkey_power_laws, morphkv, mtbbench, multi_swebench, naturebench, nester, nyu_ctf_bench, openbiorq, os_harm, perfcodebench, proofaug, pxplore, real_webagents, researchcodebench, revolve, risebench, robust_reasoning_benchmark, sib200, software_world_models, swe_smith, swebench_live, swepolybench, tau2_bench, taubench, tulu_human_eval, visual_riddles, visualwebarena, workarena.

C Reproducibility and Disclosure

Code, frozen weights, evaluation manifests, predictions, acquisition traces, and report source are available at `https://github.com/Hubert0916/arsp`. After accepting the measurement-db access conditions, run from the repository root:

```
uv sync --locked
uv run hf auth login
uv run --locked python tools/reproduce.py
```

The pipeline reconstructs the eligible inventory, fixed pools, and all 30 draws, evaluates Random acquisition and the released prediction variants on CPU, and compares probabilities within 10^{-7} and acquisition decisions exactly against the release. The submission hook is checked against the reference random rule at every candidate. Reproduction uses frozen artifacts; original offline fitting scripts are not included.

The protocol is in `experiments/formative_config.json`; the manifest records every selected item and response. `tools/score_draws.py` audits released compressed predictions without model inference, and `tools/update_formative_report.py` regenerates report tables and analyses. The release includes per-group and per-draw scores, random acquisition trajectories, and paired component differences. Earlier fixed-subject experiments remain separate historical results and are not mixed into the current tables.

File descriptions and build instructions are provided in the repository README.

Competition-specific artifacts use permitted measurement-db data and the pretrained MiniLM encoder. Codex assisted development and report preparation; no external LLM API is used during scoring.

References

Sin-Han Yang, Cheng-Kuang Wu, Chieh-Yen Lin, Yun-Nung Chen, Hung-yi Lee, and Shao-Hua Sun. On calibration of large language models: From response to capability. *arXiv preprint arXiv:2602.13540*, 2026. URL `https://arxiv.org/abs/2602.13540`.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, 1980.

Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. *arXiv preprint arXiv:2503.13335*, 2025. URL `https://arxiv.org/abs/2503.13335`.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2020. URL `https://arxiv.org/abs/2002.10957`.

AIMS Foundations. measurement-db: Public benchmark inventory and response data. Hugging Face dataset, 2026. URL `https://huggingface.co/datasets/aims-foundations/measurement-db`.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL `https://github.com/aims-foundations/paiec_baseline`.